Computer Vision and Deep Learning · Group 4

LipScribe

Technical Implementation and System Development Document

Lip-Assisted Speech Recognition for Robust Live Captioning

Team members

Cole Rodney Migisha	b29657
Charis Opol	b29654
Ainomugisha Faith	b27518
Lisa Lusinga Namugenyi	b30300
Ansasire Nehum Naamaye	b29785
Wakisa Isabelle	b30311

Student e-mail: <id>@students.ucu.ac.ug

Department of Computer Science
Uganda Christian University, Mukono, Uganda

October 2026

Status of this document. This document describes how the system proposed in the LipScribe project proposal will be implemented. It reports planned work, not completed work: no experimental results are presented, and all numerical settings that the proposal does not fix are marked as recommendations to be confirmed during development.

Contents

List of Abbreviations **iv**

1 Introduction and Implementation Overview **1**
1.1 Purpose of LipScribe . . . 1
1.2 Problem statement and objectives . . . 1
1.3 End-to-end workflow . . . 1
1.4 Scope of the initial implementation . . . 2
1.5 Structure of this document . . . 2

2 System Architecture and Design **2**
2.1 Architectural overview . . . 2
2.2 Module descriptions . . . 4
2.3 Training-time versus inference-time processing . . . 4

3 Development Environment and Technology Stack **5**
3.1 Framework decisions . . . 5
3.2 Technology stack . . . 6
3.3 Dependencies and compatibility . . . 6
3.4 Hardware requirements . . . 7
3.5 Reproducibility measures . . . 7

4 Dataset Preparation and Management **7**
4.1 The GRID audiovisual sentence corpus . . . 7
4.2 Acquisition and organisation . . . 8
4.3 Transcription normalisation and vocabulary . . . 8
4.4 Audio and video standardisation . . . 8
4.5 Training, validation and test partitions . . . 9
4.6 Dataset integrity checks . . . 9
4.7 Noise preparation with MUSAN . . . 9
4.8 Storage of processed features and metadata . . . 11

5 Visual Processing Pipeline **11**
5.1 Mouth extraction . . . 11
5.2 Visual encoder architecture . . . 12
5.3 Tensor-shape walkthrough . . . 13
5.4 Testing the visual pipeline . . . 13

6 Audio Processing Pipeline **14**
6.1 From waveform to features . . . 14
6.2 Choice of audio features . . . 15
6.3 Audio encoder . . . 15
6.4 Tensor-shape walkthrough . . . 15
6.5 Variable-length sequences and batching . . . 16
6.6 Testing the audio pipeline . . . 16

7 Multimodal Model Architecture **16**

7.1 Visual encoder . . . 17
7.2 Audio encoder . . . 17
7.3 Feature alignment and fusion . . . 17
7.4 Modality dropout . . . 19
7.5 Sequence prediction and decoding . . . 19
7.6 Forward pass and decoding algorithms . . . 21

8 Training Procedure and Experiment Management 21
8.1 Training workflow . . . 22
8.2 Initial hyperparameters . . . 22
8.3 Experiment logging and management . . . 23
8.4 Baselines and experiment plan . . . 24

9 Evaluation and Testing 25
9.1 Metrics . . . 25
9.2 Evaluation protocol . . . 26
9.3 Comparisons and their interpretation . . . 26
9.4 Ablation studies . . . 27
9.5 Latency and real-time feasibility . . . 27
9.6 Software testing . . . 27

10 Application Integration and Live Captioning 28
10.1 Batch prototype versus streaming application . . . 28
10.2 Mode A: batch captioning in Gradio . . . 29
10.3 Mode B: windowed live captioning . . . 29
10.4 Latency, cost and memory . . . 30
10.5 Testing the application . . . 30

11 Additional Objectives: Emotion Detection and Language Identification 30
11.1 Emotion detection from facial movement . . . 31
11.2 Spoken-language identification . . . 31
11.3 Summary of scope . . . 32

12 Implementation Schedule and Milestones 32

13 Risks, Limitations and Mitigation 34
13.1 Limitations of the planned system . . . 35

14 Expected Deliverables 35

15 Conclusion 36

References 37

List of Figures

1 The eight stages of the LipScribe processing workflow. 2
2 LipScribe system architecture 3
3 Threads and buffers of the windowed live-captioning mode. 29
4 Gantt-style implementation timeline. Dashed bar: optional work, started only after the core system is frozen at the end of week 5. 32

List of Tables

2 Mapping of proposal objectives to implementation components. 1
3 Module responsibilities and interfaces. 4
4 Differences between the training and inference data paths. 5
5 Technology stack **[Recommended]**. 6
6 Visual encoder (configuration V1) tensor shapes for an example batch. 13
7 Audio feature configurations. 15
8 Audio encoder tensor shapes for an example batch. 16
9 Summary of the proposed model (recommended starting configuration). 17
10 Initial training hyperparameters (starting values to be tuned). 23
11 Planned experiments. No results are reported; the last column states what each comparison is designed to reveal. 25
12 Template for reporting test WER (%). Cells are intentionally empty: no results have been obtained yet. 26
13 Representative automated tests and acceptance criteria. 28
14 Core deliverable versus optional extensions. 32
15 Milestones, dependencies, deliverables and completion criteria. 33
16 Implementation risks and mitigation strategies (L = likelihood, I = impact). . . . 34
17 Software deliverables and evidence of completion. 35

List of Algorithms

1 Mixing noise into clean speech at a target SNR. 10
2 LipScribe forward pass (training and inference). 21
3 CTC prefix beam search for one utterance (probabilities in log space in practice). 21

List of Abbreviations

Abbreviation	Meaning
AMP	Automatic Mixed Precision
ASR	Automatic Speech Recognition
AVSR	Audio-Visual Speech Recognition
BiLSTM	Bidirectional Long Short-Term Memory network
BN	Batch Normalisation
CER	Character Error Rate
CNN	Convolutional Neural Network
CTC	Connectionist Temporal Classification
DCT	Discrete Cosine Transform
FPS	Frames Per Second
GPU	Graphics Processing Unit
LM	Language Model
LSTM	Long Short-Term Memory
MFCC	Mel-Frequency Cepstral Coefficients
MHA	Multi-Head Attention
MUSAN	Music, Speech and Noise corpus
RMS	Root Mean Square
RQ	Research Question
RTF	Real-Time Factor
SNR	Signal-to-Noise Ratio
STFT	Short-Time Fourier Transform
UI	User Interface
WER	Word Error Rate

Statement labels used in this document

To separate fixed project requirements from engineering choices, statements are tagged as follows:

[Proposal] Specified by the LipScribe project proposal. These items define the intended system and are not changed without team agreement.

[Recommended] An implementation decision recommended in this document. It is a practical starting point and may be revised on evidence.

[To be decided] An open question that will be settled experimentally or by the team during development.

1 Introduction and Implementation Overview

1.1 Purpose of LipScribe

LipScribe is an audio-visual speech recognition (AVSR) system that produces text captions from a speaker's voice and lip movements together. Conventional automatic speech recognition (ASR) performs well in quiet rooms but degrades sharply when background noise, overlapping speakers or muffled speech corrupt the audio signal. Lip movements are unaffected by acoustic noise, so a model that also watches the mouth retains a usable source of evidence when the audio becomes unreliable [2, 6]. The implementation described here therefore aims to build a recogniser whose accuracy degrades more gracefully in noise than an audio-only system.

The proposal identifies deaf and hard-of-hearing people as the primary beneficiaries, with lectures, meetings and public spaces as the intended settings **[Proposal]**. These users depend on captions being correct rather than merely fast, so the implementation prioritises measurable transcription quality across noise levels and treats latency as a constraint to be measured, not assumed.

1.2 Problem statement and objectives

The proposal states the central problem as: *how can transcription be improved by combining audio understanding with lip reading so that captions remain usable even when the audio channel is severely degraded?* **[Proposal]**. It defines four objectives, each linked to a research question (RQ). Table 2 maps each objective to the part of the implementation that addresses it and states whether that part belongs to the core system.

Table 2: Mapping of proposal objectives to implementation components.

Objective (RQ)	Implementation component	Scope
1. Disambiguate homophenes (RQ1)	Cross-modal attention lets acoustic context resolve visually identical lip shapes; temporal modelling over neighbouring frames (Section 7).	Core
2. Model temporal word sequences (RQ2)	3D-CNN front-end, BiLSTM encoders and CTC training map frame sequences to whole sentences (Sections 5 to 7).	Core
3. Detect emotion from speaking style (RQ3)	Separate landmark-based classifier trained on an emotion-labelled dataset (Section 11).	Extension
4. Identify the spoken language (RQ4)	Separate language-identification head trained on multilingual data (Section 11).	Extension

1.3 End-to-end workflow

At run time the system executes the same sequence of stages for every segment of input. Figure 1 summarises them; Section 2 describes each stage as a software module.

1. **Capture** synchronised video frames and audio samples from a camera and microphone, or read them from a recorded clip.

2. **Detect the face** in every frame with MediaPipe Face Mesh [15, 14] and locate the lip landmarks.

3. **Extract the mouth sequence** by cropping a fixed-size region around the lips in each frame.
4. **Extract audio features** (log-Mel spectrograms, optionally MFCCs) from the waveform.
5. **Encode each modality** with its own neural encoder: a 3D-CNN and BiLSTM for video, and a BiLSTM for audio.
6. **Fuse** the two encoded sequences with cross-modal attention.
7. **Decode** the fused sequence into characters with a CTC output layer and beam search.
8. **Display** the resulting text as a caption.



Figure 1: The eight stages of the LipScribe processing workflow.

1.4 Scope of the initial implementation

The core deliverable is the audio-visual speech recogniser and a captioning demonstration built on it. It covers objectives 1 and 2 and the evaluation plan in the proposal: training on the GRID corpus [9] with MUSAN noise [10] added at clean, 10, 5, 0 and −5 dB SNR, and reporting WER and CER on held-out speakers **[Proposal]**.

Emotion detection and language identification (objectives 3 and 4) are not part of the core architecture in this plan. GRID provides no emotion labels and contains a single language, so neither task can be trained or evaluated on it alone. Section 11 describes how each could be investigated as a separate module with its own data and evaluation, to be attempted only after the core system works (Section 12). Keeping them separate prevents unvalidated extensions from delaying or obscuring the main result.

1.5 Structure of this document

Sections 2 and 3 define the system architecture and technology stack. Sections 4 to 6 describe the data and the two preprocessing pipelines. Sections 7 to 9 specify the model, its training and its evaluation. Section 10 covers the live captioning application, Section 11 the optional extensions, and Sections 12 to 14 the schedule, risks and deliverables.

2 System Architecture and Design

2.1 Architectural overview

LipScribe is organised as a pipeline of independent modules connected by well-defined tensor interfaces. Two parallel streams process the visual and audio inputs; a fusion module combines their encoded sequences; and a CTC-based head turns the fused sequence into text. Figure 2 shows the complete architecture. Each module can be developed, tested and replaced on its own, which lets team members work in parallel and lets the baselines in Section 8 reuse the same components.

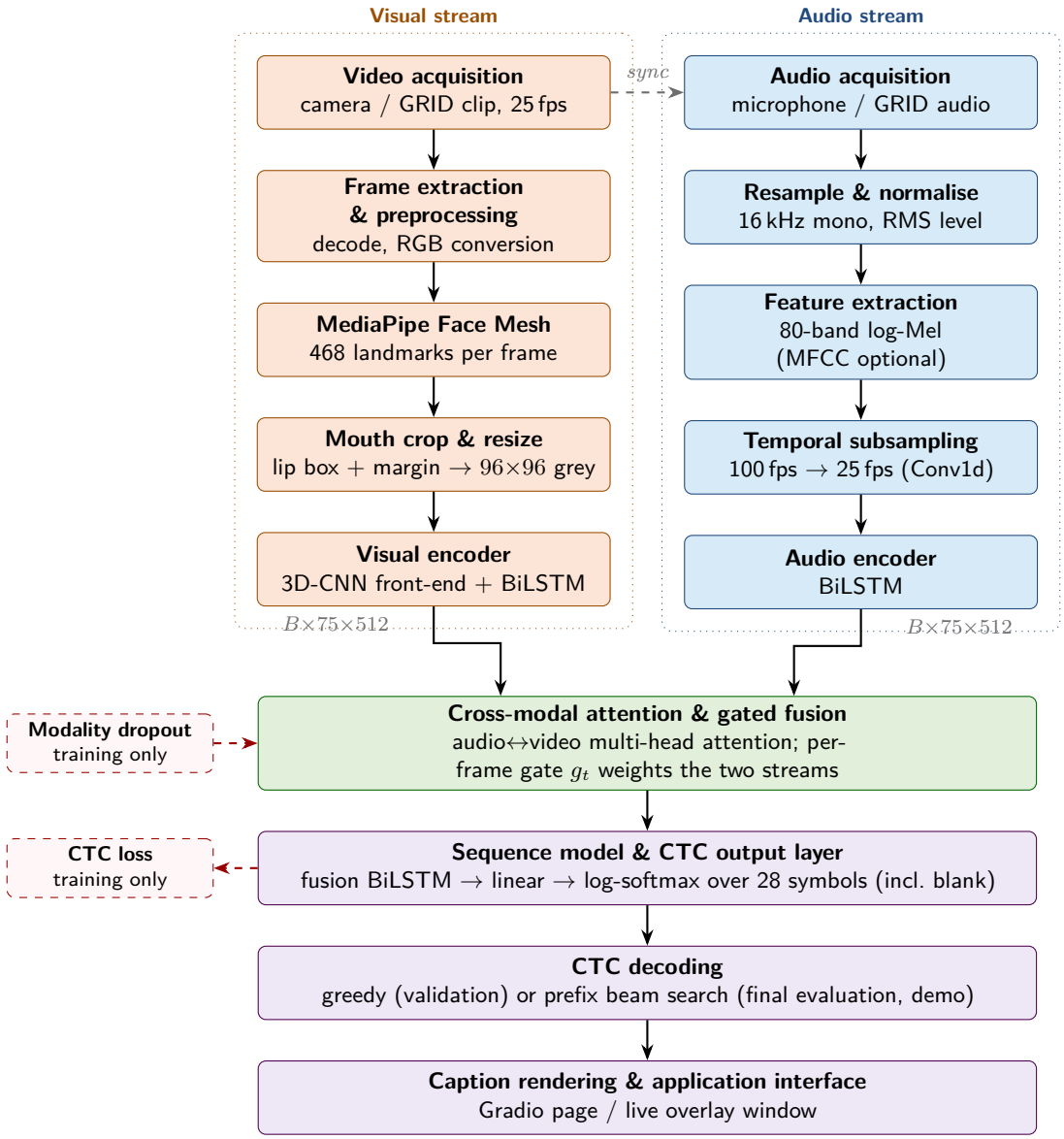


Figure 2: LipScribe system architecture. The visual (left) and audio (right) streams are processed independently, fused by cross-modal attention and decoded with CTC. Dashed red components are active only during training. Tensor sizes assume a 3 s GRID clip ($T = 75$ video frames) and the recommended configuration of Section 7.

2.2 Module descriptions

Table 3 lists every module with its inputs, outputs and connections. Interfaces are expressed as tensor shapes so that each module can be tested in isolation; B is the batch size, T the number of video frames, and T_a the number of audio feature frames.

Table 3: Module responsibilities and interfaces.

Module	Purpose	Input	Output	Connects to
A/V acquisition	Read synchronised frames and samples from a file or device	Video file / camera, audio file / microphone	Frame list with timestamps; waveform	Frame extraction, resampling
Frame extraction	Decode frames, convert colour space, enforce frame rate	Raw frames	RGB frames $\langle T, H, W, 3\rangle$	Face Mesh
MediaPipe Face Mesh	Locate 468 facial landmarks per frame [15]	RGB frame	Landmark array $\langle 468, 3\rangle$ or "not found"	Mouth crop
Mouth crop & resize	Extract a stable, fixed-size lip region	Frame and lip landmarks	Grey crop $\langle 96, 96\rangle$, validity flag	Visual encoder
Visual encoder	Model lip appearance and motion over time	Clip $\langle B, 1, T, 96, 96\rangle$	Sequence $\langle B, T, 512\rangle$	Fusion
Audio resample & normalise	Standardise sample rate, channels and level	Waveform at source rate	16 kHz mono waveform	Feature extraction
Feature extraction	Compute time–frequency representation	Waveform $\langle N\rangle$	Log-Mel $\langle T_a, 80\rangle$, $T_a \approx 4T$	Audio encoder
Audio encoder	Subsample to video rate and model acoustic context	Features $\langle B, T_a, 80\rangle$	Sequence $\langle B, T, 512\rangle$	Fusion
Attention fusion	Combine streams, weighting the more reliable one	Two sequences $\langle B, T, 512\rangle$	Fused sequence $\langle B, T, 256\rangle$	CTC head
CTC head	Produce per-frame character distributions	Fused sequence	Log-probabilities $\langle T, B, 28\rangle$	Loss / decoder
CTC decoder	Convert frame posteriors to text	Log-probabilities	Character string	Caption UI
Caption UI	Display text and handle user interaction	String, timing information	Rendered caption	User

2.3 Training-time versus inference-time processing

The same model code runs in training and at inference, but the surrounding data path differs. Separating the two paths early avoids a common failure in which preprocessing at demonstration time silently diverges from the preprocessing the model was trained on.

Table 4: Differences between the training and inference data paths.

Aspect	Training (offline)	Inference (demo)
Visual preprocessing	Mouth crops pre-computed once and cached to disk	Face Mesh and cropping run on every incoming frame
Audio input	Clean GRID audio mixed with MUSAN noise on the fly at a sampled SNR	Raw microphone or clip audio; noise is whatever the room contains
Augmentation	Random crop jitter, horizontal flip, modality dropout	None
Normalisation	Statistics computed on the training split	Same stored statistics loaded from file
Output	CTC loss computed against the transcript	Beam-search decoding to text
Unit of processing	Complete 3 s utterance	Fixed-length window from a rolling buffer (Section 10)
Model mode	`model.train()`: dropout and batch-norm updates active	`model.eval()` under `torch.inference_mode()`

All preprocessing functions that both paths need—mouth cropping, feature extraction, normalisation and token mapping—live in a single shared package (`lipscribe/preprocessing`). The training scripts and the demonstration import the same functions, and an integration test (Section 9.6) checks that a GRID clip processed through the live path produces the same tensors as the cached training features.

3 Development Environment and Technology Stack

3.1 Framework decisions

The team will implement LipScribe in Python and select one tool for each role to keep the codebase small and the learning burden manageable. Two choices shape the rest of the project.

Deep-learning framework: PyTorch [Recommended]. PyTorch provides every model component the proposal requires—3D convolutions (`nn.Conv3d`), bidirectional LSTMs with packed variable-length sequences, multi-head attention (`nn.MultiheadAttention`) and a built-in CTC loss (`nn.CTCLoss`). Its eager execution makes tensor shapes easy to inspect during debugging, and `torchaudio` supplies log-Mel and MFCC transforms that run identically on CPU and GPU. `torchvision` supplies an ImageNet-pretrained EfficientNet-B0 [13] for the backbone experiment in Section 5. TensorFlow could serve equally well; the team chooses PyTorch so that one framework covers training, evaluation and inference.

Interface framework: Gradio [Recommended]. Gradio builds a browser-based demonstration from Python functions, with ready-made webcam, microphone and video-upload components. This matches the "simple web demonstration" promised in the proposal **[Proposal]** without requiring front-end development. The lower-latency live mode (Section 10) uses an OpenCV display window driven by the same inference code, because it avoids browser round-trips; it is a development tool rather than a second interface framework.

3.2 Technology stack

Table 5 lists each technology, its purpose and where it is used.

Table 5: Technology stack **[Recommended]**.

Technology	Purpose	System component
Python 3.11	Implementation language for all modules	Entire system
PyTorch, torchaudio, torchvision	Model definition, training, CTC loss, audio features, pretrained EfficientNet-B0	Encoders, fusion, training, inference
OpenCV (`opencv-python`)	Video decoding, camera capture, cropping, resizing, live display	Visual pipeline, live demo
MediaPipe	Face Mesh landmark detection [14, 15]	Mouth localisation
NumPy	Array manipulation and on-disk storage of cached crops	Preprocessing, data loading
pandas	Dataset manifest (one row per utterance), split tables, result tables	Data management, evaluation
soundfile	Reading and writing WAV audio	Audio pipeline, noise mixing
librosa	Reference implementation used to cross-check feature extraction in tests	Unit tests
sounddevice	Microphone capture for the live loop	Live demo
scikit-learn	Speaker-grouped splitting (`GroupShuffleSplit`), confusion matrices for extensions	Data splits, extensions
jiwer	Word and character error rate computation	Evaluation
Matplotlib	Training curves, WER-versus-SNR plots, attention-gate plots	Experiment analysis
TensorBoard	Live logging of losses, error rates and learning rate	Training
PyYAML	Experiment configuration files	All scripts
pytest	Unit, integration and end-to-end tests	Quality assurance
Gradio	Web demonstration interface	Application
Git and GitHub	Version control and collaboration	Project management

3.3 Dependencies and compatibility

Three compatibility issues need attention when the environment is first created.

- **MediaPipe and Python versions.** MediaPipe publishes wheels for a limited range of Python versions. The team will confirm that the chosen Python version has a MediaPipe wheel for every member's operating system before fixing the version; Python 3.11 is proposed because it is widely supported **[To be decided]**.
- **PyTorch and CUDA.** The PyTorch build must match the CUDA driver of whichever GPU is used. The CPU-only build is sufficient for preprocessing, tests and the demonstration.
- **Paired library versions.** `torch`, `torchaudio` and `torchvision` must be installed as a matching set.

3.4 Hardware requirements

The project has not yet established which hardware is available. The plan therefore states requirements rather than assuming a particular machine.

- **Training** needs a CUDA-capable GPU; training the full model on a CPU alone is unlikely to finish within the six-week schedule. Candidate sources are a university GPU workstation, a team member's GPU, or a cloud notebook service **[To be decided]**. With a GPU of about 8 GB memory, the recommended batch size of 16 clips with mixed precision is expected to fit; this will be confirmed by a memory test in week 2.
- **Storage.** Cached mouth crops for the whole corpus occupy roughly 23 GB as uncompressed 8-bit arrays ($\approx 33{,}000 \times 75 \times 96 \times 96$ bytes), plus the raw corpus and MUSAN. About 100 GB of free disk space is recommended.
- **Demonstration** requires a laptop with a webcam and microphone. A GPU is helpful but not required; latency will be measured on the actual demonstration machine.

3.5 Reproducibility measures

- Exact dependency versions are pinned in `requirements.txt` (and an equivalent `environment.yml`), generated from a working environment rather than written by hand.
- Every experiment is defined by a YAML configuration file committed to Git; the configuration and Git commit hash are saved inside each checkpoint.
- Random seeds for Python, NumPy and PyTorch are set from the configuration, and the noise-mixing seed for the test set is fixed so that every model is evaluated on identical noisy audio.
- The speaker split is stored as a file (`splits.csv`) and loaded by every script, never regenerated.
- Preprocessing scripts are deterministic and record the MediaPipe version and crop parameters in the dataset manifest.

4 Dataset Preparation and Management

4.1 The GRID audiovisual sentence corpus

The proposal selects the GRID corpus as the primary dataset **[Proposal]**. GRID contains audio and frontal-face video recordings of 34 talkers, each speaking 1000 short sentences [9]; the proposal quotes approximately 33,000 usable sentences. Every sentence follows the same six-word grammar,

`command colour preposition letter digit adverb`,

as in "place green at B 4 now" [9]. The vocabulary is therefore small and closed, which makes GRID suitable for a six-week project but also limits how far conclusions generalise to open conversation (Section 13). Each utterance is distributed with a word-level alignment file giving start and end times, from which the transcript is derived.

4.2 Acquisition and organisation

The team will download the video, audio and alignment archives for every speaker from the official distribution and record the download date and archive checksums in `data/raw/SOURCES.md`. The raw files are never modified. All derived data are written to a separate tree:

Listing 1: Proposed data directory layout **[Recommended]**.

```
data/
  raw/grid/s1/video/*.mpg s1/audio/*.wav s1/align/*.align
  raw/musan/noise/...
  processed/
    mouths/s1/<utt_id>.npy # uint8 [T, 96, 96]
    audio16k/s1/<utt_id>.wav # 16 kHz mono, clean
    noisy_test/<snr>/<utt_id>.wav # fixed noisy copies for evaluation
  manifest.csv # one row per utterance
  splits.csv # speaker -> train / val / test
  stats/feature_norm.npz # training-set mean and std
```

The manifest is the single index of the dataset. Each row records the utterance identifier, speaker, transcript, number of video frames, audio duration, the fraction of frames with a successful face detection, and the paths of all derived files. Scripts select data by filtering the manifest, never by listing directories.

4.3 Transcription normalisation and vocabulary

Transcripts are built from the alignment files by removing the silence and short-pause tokens and joining the remaining words with single spaces. Text is lower-cased and restricted to the characters `a`–`z` and space **[Recommended]**. The letter slot is spelled as a single character ("b") and digits as words ("four"), matching the alignment files. The output vocabulary is therefore

$$\mathcal{V} = \{\texttt{<blank>}, \sqcup, \texttt{a}, \ldots, \texttt{z}\}, \qquad |\mathcal{V}| = 28, \tag{1}$$

with the CTC blank at index 0. A character vocabulary is chosen over a word vocabulary because it keeps CER meaningful, lets the same model architecture be reused for other text later, and matches the CTC systems cited in the proposal [4, 6].

4.4 Audio and video standardisation

- **Audio.** All audio is converted to 16 kHz, 16-bit, mono WAV **[Recommended]**. 16 kHz is the standard rate for speech recognition, preserves the frequency range relevant to speech, and matches the rate the microphone path will deliver in the demonstration. Resampling uses a band-limited (anti-aliasing) resampler.
- **Video.** GRID video is recorded at 25 frames per second, and a 3 s utterance nominally yields 75 frames. Frames are decoded with OpenCV and their count checked against the audio duration. The training pipeline assumes exactly 25 fps; the live pipeline resamples camera frames to 25 fps by timestamp (Section 10).
- **Synchronisation.** Audio and video of a GRID utterance start together. Sample n of the 16 kHz waveform corresponds to video frame $\lfloor 25n/16000 \rfloor$, so one video frame spans 640 audio samples and four 10 ms audio feature frames. This fixed 4:1 ratio is used throughout Sections 6 and 7.

4.5 Training, validation and test partitions

The proposal requires evaluation on held-out speakers **[Proposal]**. Partitions are therefore made by speaker, not by utterance: every utterance from a given speaker belongs to exactly one partition. The recommended split **[Recommended]** is

- **Test:** 4 speakers (2 female, 2 male), used only for final reporting;
- **Validation:** 3 speakers, used for early stopping and model selection;
- **Training:** all remaining speakers (about 27).

Speakers with missing or badly corrupted video, as identified by the integrity checks below, are excluded before the split is drawn. The split is generated once with a fixed seed, reviewed for gender balance, written to `splits.csv` and committed to the repository.

Why speaker leakage matters. If utterances from one speaker appear in both training and test data, the model can partly memorise that speaker's lip shape, face and voice. Test error then measures recognition of familiar people rather than generalisation to new users—the situation that matters for a captioning tool. Speaker leakage would make every reported WER optimistically biased. Three safeguards prevent it: the split file is the only source of partition membership; the data loader raises an error if a speaker appears in more than one partition; and an automated test (Section 9.6) asserts that the three speaker sets are disjoint. Normalisation statistics are computed on the training partition only, for the same reason.

4.6 Dataset integrity checks

A script `verify_dataset.py` runs before any training and writes a report. It checks that:

1. every utterance has a video file, an audio file and an alignment file;
2. video decodes without error and has 75 ± 2 frames;
3. audio duration matches video duration to within one frame (40 ms);
4. the transcript contains exactly six words drawn from the GRID word list;
5. Face Mesh succeeds on at least 90% of frames **[Recommended]**; utterances below this threshold are flagged and excluded;
6. no audio file is silent or clipped throughout (RMS and peak checks).

The report records how many utterances pass each check. These counts become the dataset statistics quoted in the final report; no statistics are assumed in advance.

4.7 Noise preparation with MUSAN

The proposal adds synthetic noise from the MUSAN corpus [10] at five conditions: clean, 10 dB, 5 dB, 0 dB and -5 dB SNR **[Proposal]**. MUSAN contains separate music, speech and noise subsets. The core experiments use the *noise* subset **[Recommended]**, which matches the proposal's "background noise" scenario; babble noise built from the speech subset is a possible later experiment.

Noise partitions. MUSAN noise files are split into disjoint training, validation and test pools by file **[Recommended]**. Using different noise recordings at test time ensures that the evaluation measures robustness to unseen noise rather than recognition of memorised noise patterns.

Mixing procedure. Let $s[n]$ be the clean speech waveform of length N and $\tilde{w}[n]$ a noise segment of the same length, cut at a random offset from a noise file (looped if the file is shorter than N). The signal powers are

$$P_s = \frac{1}{N}\sum_{n=1}^{N} s[n]^2, \qquad P_w = \frac{1}{N}\sum_{n=1}^{N} \tilde{w}[n]^2. \tag{2}$$

To obtain a target SNR of $\rho\,\mathrm{dB}$, the noise is scaled by

$$\alpha = \sqrt{\frac{P_s}{P_w \cdot 10^{\rho/10}}}, \qquad y[n] = s[n] + \alpha\,\tilde{w}[n], \tag{3}$$

which gives $10\log_{10}(P_s/(\alpha^2 P_w)) = \rho$ exactly. If the peak $\max_n |y[n]|$ exceeds 1.0, the whole mixture y is scaled down; because speech and noise are scaled together, the SNR is unchanged. Algorithm 1 summarises the procedure.

Algorithm 1 Mixing noise into clean speech at a target SNR.

Input: clean speech s (length N), noise pool $\mathcal{W}$, target SNR ρ (dB), random generator r
Output: noisy speech y, measured SNR $\hat{\rho}$

if $\rho =$ clean **then return** s, ∞
end if
$w \leftarrow$ file drawn from $\mathcal{W}$ using r
$\tilde{w} \leftarrow$ segment of length N from w at random offset (loop w if shorter)
$P_s \leftarrow \mathrm{mean}(s^2)$; $\quad P_w \leftarrow \mathrm{mean}(\tilde{w}^2)$
$\alpha \leftarrow \sqrt{P_s/(P_w \cdot 10^{\rho/10})}$
$y \leftarrow s + \alpha\tilde{w}$
$\hat{\rho} \leftarrow 10\log_{10}(P_s/\,\mathrm{mean}((\alpha\tilde{w})^2))$ ▷ verification
assert $|\hat{\rho} - \rho| < 0.01$
if $\max|y| > 1$ **then** $y \leftarrow y/\max|y|$ ▷ preserves SNR
end if
return y, $\hat{\rho}$

Speech power definition. Equation (2) measures speech power over the whole clip, including the short silences at the start and end of each GRID utterance **[Recommended]**. This definition is simple and reproducible. Its drawback is that the SNR during active speech is somewhat higher than the nominal value. The final report will state this definition explicitly; measuring power over voiced frames only is an alternative that the team may adopt if consistency with a particular reference paper becomes important **[To be decided]**.

Training versus test noise. During training, noise is mixed on the fly: for each utterance an SNR is drawn from $\{\text{clean}, 10, 5, 0, -5\}$ dB with equal probability **[Recommended]**, so the model sees each utterance under many conditions over the course of training. For validation

and testing, each utterance is mixed once per SNR with a fixed seed and the result is stored in `processed/noisy_test/`. Every model is then evaluated on exactly the same audio.

Labels under noise. Adding noise changes only the audio waveform. The video and transcript of the utterance are unchanged, so every noisy copy keeps the transcript of its clean original. This property is what allows the same test sentences to be scored at every SNR and the results compared directly.

4.8 Storage of processed features and metadata

Mouth crops are stored as one uncompressed `uint8` NumPy array per utterance **[Recommended]**, which allows fast memory-mapped loading; grey-scale storage reduces size threefold compared with colour. Clean audio is stored as 16 kHz WAV, and log-Mel features are computed on the fly on the GPU, because the noise—and therefore the features—changes every time an utterance is loaded during training. Feature normalisation statistics (per-band mean and standard deviation of the log-Mel features over clean and noisy training audio) are stored in `stats/feature_norm.npz` and loaded by both training and inference code.

5 Visual Processing Pipeline

The visual pipeline converts a video clip into a temporally ordered sequence of fixed-size mouth images and then into a sequence of visual feature vectors. Preprocessing (Section 5.1) runs once per utterance and is cached; the encoder (Section 5.2) is part of the trained model.

5.1 Mouth extraction

1. Reading frames. Frames are decoded in order with `cv2.VideoCapture` and converted from BGR to RGB, the format MediaPipe expects. Each frame keeps its index t, which is the only time reference used for GRID; at 25 fps, frame t starts at $40t$ ms.

2. Detecting landmarks. MediaPipe Face Mesh returns 468 three-dimensional landmarks per detected face [15, 3]. It is run in video mode with `max_num_faces=1`, so that landmarks are tracked from frame to frame instead of being re-detected independently, which is both faster and more temporally stable.

3. Selecting lip landmarks. The Face Mesh topology defines a fixed subset of landmarks on the outer and inner lip contours. The implementation takes these indices from MediaPipe's own lip-connection definition (`FACEMESH_LIPS`) rather than hard-coding them, which avoids errors if the index list is mistyped.

4. Computing the mouth box. Let (x_i, y_i) be the pixel coordinates of the lip landmarks in frame t. The mouth centre and size are

$$c_t = \Big(\tfrac{\min_i x_i + \max_i x_i}{2},\ \tfrac{\min_i y_i + \max_i y_i}{2}\Big), \qquad S_t = \max\big(\max_i x_i - \min_i x_i,\ \max_i y_i - \min_i y_i\big). \tag{4}$$

A square box is used so that resizing does not distort the mouth's aspect ratio.

5. Adding a margin and stabilising. The box side is enlarged to $L_t = (1 + 2m)\, S_t$ with a margin $m = 0.25$ **[Recommended]**, so that the chin, the corners of the mouth and jaw movement remain visible when the mouth opens. Raw per-frame boxes jitter by a few pixels, and this jitter would appear to the network as spurious motion. The centre and side length are therefore smoothed over time with a centred moving average of five frames **[Recommended]**. For offline preprocessing a centred window is possible because the whole clip is available; the live pipeline uses an exponential moving average over past frames instead.

6. Cropping and resizing. The square region of side L_t centred on c_t is cropped (with edge padding if it extends beyond the frame) and resized to 96×96 pixels using area interpolation. The proposal allows 96×96 or 112×112 **[Proposal]**; 96×96 is chosen **[Recommended]** because it reduces input size by about 27% and is comparable to the mouth-crop sizes used in earlier sentence-level lipreading work on GRID [12]. 112×112 remains an option if the EfficientNet-B0 experiment benefits from it.

7. Normalising. Crops are converted to grey-scale and stored as 8-bit integers. At load time they are scaled to $[0, 1]$ and standardised with the mean and standard deviation of all training crops. Grey-scale input removes colour differences between speakers and cameras, which carry little information about speech.

8. Forming sequences. The crops of one utterance are stacked in frame order into an array of shape $\langle T, 96, 96\rangle$. Temporal order is never changed: data augmentation applies the same random crop offset and the same horizontal flip to every frame of a clip, so the motion pattern is preserved.

9. Handling failures. Three cases are handled explicitly.

- **Occasional failed detections.** If Face Mesh fails on a frame, its box is linearly interpolated from the nearest successful frames on either side, and the frame is marked invalid in a per-frame mask.
- **Long runs of failures.** If more than 10% of frames fail **[Recommended]**, the utterance is excluded from training and listed in the integrity report. In the live demo, the visual stream for that window is treated as missing (Section 7.4).
- **Inconsistent lengths.** Clips are padded or truncated to the audio-derived frame count so that both streams describe the same time span; padding repeats the last valid frame and is marked in the mask.

Alignment with audio. Because every crop retains its original frame index and audio sample n maps to frame $\lfloor 25n/16000 \rfloor$, visual frame t is aligned with audio feature frames $4t$ to $4t + 3$. This alignment is preserved by never dropping frames: failed frames are interpolated, not removed.

5.2 Visual encoder architecture

The proposal specifies a 3D convolutional front-end followed by a bidirectional LSTM **[Proposal]**, and its literature discussion also names EfficientNetB0 as an efficient 2D backbone [4, 5]. These

are two different ways of extracting per-frame spatial features, so this plan defines them as two clearly separate configurations rather than stacking both in one network.

Primary configuration V1: 3D-CNN + BiLSTM [Recommended]. Three 3D convolution blocks process the clip. Each kernel spans several consecutive frames, so every output feature responds both to the mouth's shape and to how it changes over a short time window [1, 12]. All temporal strides are 1, which keeps one feature vector per video frame; only the spatial dimensions are downsampled. The spatial map of each frame is then flattened, projected to 512 dimensions, and passed to a two-layer BiLSTM that models longer-range context in both directions [4]. This configuration is the proposal's "3D convolutional visual encoder" advanced technique and is used in all main experiments.

Alternative configuration V2: 3D stem + EfficientNet-B0 trunk [To be decided]. A single 3D convolution captures short-term motion and outputs a 3-channel map per frame. Each frame's map is then passed independently through an ImageNet-pretrained EfficientNet-B0 [13] (by folding time into the batch dimension), globally average-pooled to a 1280-dimensional vector, projected to 512 dimensions and fed to the same BiLSTM. This mirrors the common "3D front-end plus 2D trunk" design [6] and tests whether transfer learning from a pretrained 2D backbone helps on GRID. Because EfficientNet-B0 is applied to all 75 frames of every clip, V2 is several times more expensive than V1; it is run as an ablation only if time and compute allow (Section 12).

5.3 Tensor-shape walkthrough

Table 6 traces one batch through configuration V1. The example assumes a batch of $B = 16$ clips, $T = 75$ frames and 96×96 grey-scale crops; the channel counts are recommended starting values **[Recommended]**. Shapes follow PyTorch's $\langle B, C, T, H, W \rangle$ convention for 3D convolutions.

Table 6: Visual encoder (configuration V1) tensor shapes for an example batch.

Stage	Operation	Output shape
Input	Normalised grey mouth crops	$\langle 16, 1, 75, 96, 96 \rangle$
Block 1	Conv3d(1→32, kernel 3×5×5, stride 1×2×2), BN, ReLU	$\langle 16, 32, 75, 48, 48 \rangle$
	MaxPool3d(kernel and stride 1×2×2)	$\langle 16, 32, 75, 24, 24 \rangle$
Block 2	Conv3d(32→64, kernel 3×5×5, stride 1), BN, ReLU, MaxPool 1×2×2	$\langle 16, 64, 75, 12, 12 \rangle$
Block 3	Conv3d(64→96, kernel 3×3×3, stride 1), BN, ReLU, MaxPool 1×2×2	$\langle 16, 96, 75, 6, 6 \rangle$
Reshape	Move time forward and flatten each frame: $96 \cdot 6 \cdot 6 = 3456$	$\langle 16, 75, 3456 \rangle$
Projection	Linear(3456→512), ReLU, Dropout(0.3)	$\langle 16, 75, 512 \rangle$
Temporal	2-layer BiLSTM, 256 units per direction	$\langle 16, 75, 512 \rangle$

Temporal padding of 1 frame on each side of every convolution keeps the time dimension at exactly 75, so the visual encoder emits one 512-dimensional vector per video frame. The last dimension of the output, 512, is the concatenation of the forward and backward LSTM states.

5.4 Testing the visual pipeline

- A unit test runs the crop function on a stored GRID frame and asserts an output of shape $(96, 96)$, type `uint8`, and a box centre inside the lower half of the face bounding box.

- A visual check script writes a contact sheet of crops for a random sample of utterances from every speaker; a team member inspects it for mis-centred or blurred crops before training starts.
- A unit test passes a random tensor of shape $\langle 2, 1, 75, 96, 96\rangle$ through the encoder and asserts an output of $\langle 2, 75, 512\rangle$ with finite values.

6 Audio Processing Pipeline

The audio pipeline turns a waveform into a sequence of acoustic feature vectors at the video frame rate. Unlike the visual crops, audio features are computed on the fly during training, because noise is mixed into each utterance afresh every time it is loaded (Section 4.7).

6.1 From waveform to features

Loading and resampling. Waveforms are read with `soundfile` as 32-bit floating-point arrays in $[-1, 1]$. Stereo input is averaged to mono, and any input that is not at 16 kHz is resampled with `torchaudio.functional.resample`. GRID audio is resampled once during preprocessing; microphone audio is captured at 16 kHz directly where the device allows it.

Amplitude normalisation. The waveform presented to the model—the noisy mixture during training, the raw microphone signal at inference—is scaled to a fixed RMS level of -25 dBFS **[Recommended]**, with the gain limited to $+20$ dB so that near-silence is not amplified into loud noise. This removes differences in recording level between GRID speakers and the demonstration microphone. Normalisation is applied *after* noise mixing so that it does not alter the SNR.

Framing and windowing. The signal is split into 25 ms frames (400 samples) with a 10 ms hop (160 samples), and each frame is multiplied by a Hann window to reduce spectral leakage **[Recommended]**. These values are the conventional choice for speech, because speech is approximately stationary over 20–30 ms. The 10 ms hop yields 100 feature frames per second, exactly four per video frame.

Log-Mel spectrogram. A 512-point short-time Fourier transform (STFT) gives the power spectrum $|X_k(\tau)|^2$ of each frame τ. A bank of $M = 80$ triangular Mel filters $H_m(k)$ spanning 20 Hz–8 kHz **[Recommended]** is applied, and the result is log-compressed:

$$\mathrm{LM}_m(\tau) = \log\Big(\epsilon + \sum_k H_m(k)\,|X_k(\tau)|^2\Big), \qquad m = 1, \ldots, 80, \quad \epsilon = 10^{-6}. \tag{5}$$

The Mel scale spaces filters more densely at low frequencies, mirroring human pitch perception, and the logarithm compresses the large dynamic range of speech energy.

MFCCs. Mel-frequency cepstral coefficients are obtained by applying a discrete cosine transform (DCT) to the log-Mel vector and keeping the first 13 coefficients, optionally with their first and second temporal derivatives (Δ, $\Delta\Delta$) to give 39 values per frame:

$$c_j(\tau) = \sum_{m=1}^{M} \mathrm{LM}_m(\tau)\cos\Big[\frac{\pi j}{M}\Big(m - \tfrac{1}{2}\Big)\Big], \qquad j = 0, \ldots, 12. \tag{6}$$

Feature normalisation. Each feature dimension is standardised with a mean and standard deviation estimated on the training partition, under the same SNR distribution used in training. The statistics are computed once by `compute_stats.py`, stored in `stats/feature_norm.npz` and loaded unchanged at validation, test and inference time. Statistics are never computed from validation or test data.

6.2 Choice of audio features

The proposal lists both log-Mel spectrograms and MFCCs **[Proposal]** but does not state whether they are used together. Because MFCCs are a linear transform (the DCT) of the log-Mel features, a neural encoder given log-Mel input can in principle learn any information that MFCCs contain. The plan therefore uses a manageable initial configuration and treats the others as experiments:

Table 7: Audio feature configurations.

ID	Features (dimension)	Role
A1	80 log-Mel	Primary configuration for all models **[Recommended]**
A2	13 MFCC + Δ + $\Delta\Delta$ (39)	Experiment: compact, decorrelated input
A3	log-Mel $\oplus$ MFCC (119)	Experiment: the combined input named in the proposal

All three configurations are computed by the same `torchaudio` transform objects, so switching between them is a one-line configuration change. A2 and A3 are run on the audio-only baseline first, because it trains fastest; the best configuration is then adopted for the fused model if it improves validation WER.

6.3 Audio encoder

The audio encoder maps the 100 fps features to a 25 fps sequence of acoustic representations aligned with the visual sequence. Two 1-D convolutions with stride 2 reduce the frame rate by a factor of four while mixing information across neighbouring frames. A two-layer BiLSTM, the encoder specified in the proposal **[Proposal]**, then models context across the whole utterance. Strided convolutions are preferred to simply discarding frames, because they learn which local acoustic detail to retain.

For a convolution with kernel $k = 3$, padding $p = 1$ and stride $s = 2$, an input of length L produces an output of length $\lfloor (L + 2p - k)/s \rfloor + 1 = \lceil L/2 \rceil$. Two such layers turn T_a audio frames into $\lceil T_a/4 \rceil$ encoder frames. The same formula is used in the data loader to compute per-utterance output lengths for CTC (Section 7.5).

6.4 Tensor-shape walkthrough

Table 8 assumes a batch of $B = 16$ clips of 3 s at 16 kHz ($N = 48{,}000$ samples) and configuration A1. The STFT with centred frames produces $N/160 + 1 = 301$ frames; the last frame is dropped so that $T_a = 300 = 4T$ **[Recommended]**.

Table 8: Audio encoder tensor shapes for an example batch.

Stage	Operation	Output shape
Waveform	Normalised 16 kHz mono audio	$\langle 16, 48000 \rangle$
Features	Log-Mel (Eq. (5)), trim to $4T$, standardise	$\langle 16, 300, 80 \rangle$
Transpose	Channels-first layout for Conv1d	$\langle 16, 80, 300 \rangle$
Subsample 1	Conv1d(80→256, k=3, s=2), BN, ReLU	$\langle 16, 256, 150 \rangle$
Subsample 2	Conv1d(256→256, k=3, s=2), BN, ReLU	$\langle 16, 256, 75 \rangle$
Transpose	Time-first layout for LSTM	$\langle 16, 75, 256 \rangle$
Temporal	2-layer BiLSTM, 256 units per direction, dropout 0.3	$\langle 16, 75, 512 \rangle$

The dimensions are: batch (B), time (audio frames at 100 fps before subsampling, encoder frames at 25 fps after), and features (Mel bands, then convolution channels, then concatenated forward and backward LSTM states). The output has the same shape as the visual encoder output, which simplifies fusion.

6.5 Variable-length sequences and batching

Most GRID utterances have the same 3 s duration, but some are shorter, and windows in the live demo may vary. The collate function therefore

1. pads every waveform in a batch with zeros to the longest one,
2. records the true number of feature frames $T_a^{(b)}$ for each item b,
3. computes encoder lengths $T^{(b)} = \lceil T_a^{(b)}/4 \rceil$, and
4. builds a Boolean padding mask of shape $\langle B, T \rangle$.

The BiLSTM receives the lengths through `pack_padded_sequence`, so padded frames do not contaminate the backward direction; the fusion attention ignores padded keys through the mask; and CTC receives the true lengths. Utterances are grouped into batches of similar length (bucketing) when lengths vary widely, to reduce wasted computation on padding.

6.6 Testing the audio pipeline

- The log-Mel output of `torchaudio` is compared with `librosa.feature.melspectrogram` on a test file using identical parameters; the maximum absolute difference in log-energy must be below a small tolerance (for example 10^{-3}).
- A 3 s waveform must produce exactly 300 feature frames and 75 encoder frames.
- Normalised training features must have per-band mean within 0.05 of 0 and standard deviation within 0.05 of 1.
- The audio encoder on a random input of $\langle 2, 300, 80 \rangle$ returns $\langle 2, 75, 512 \rangle$.

7 Multimodal Model Architecture

This section specifies the complete recognition model in enough detail to be implemented directly. It combines the encoders of Sections 5 and 6 with a fusion module and a CTC output

layer. Table 9 gives the recommended dimensions **[Recommended]**; all are starting values to be tuned.

Table 9: Summary of the proposed model (recommended starting configuration).

Component	Configuration	Output
Visual encoder	3 Conv3d blocks (32/64/96 ch.), Linear 512, 2-layer BiLSTM (256/dir.)	$\langle B, T, 512\rangle$
Audio encoder	2 strided Conv1d (256 ch.), 2-layer BiLSTM (256/dir.)	$\langle B, T, 512\rangle$
Projections	Linear 512→256 per stream, LayerNorm	$\langle B, T, 256\rangle$ each
Cross-modal attention	Two MHA blocks (A→V, V→A), 4 heads, $d = 256$	$\langle B, T, 256\rangle$ each
Gated fusion	Per-frame scalar gate g_t	$\langle B, T, 256\rangle$
Sequence model	1-layer BiLSTM (128/dir.)	$\langle B, T, 256\rangle$
CTC output layer	Linear 256→28, log-softmax	$\langle T, B, 28\rangle$

7.1 Visual encoder

Purpose. Convert a mouth-crop sequence into one feature vector per video frame that describes the lip configuration and its motion. **Input.** $\langle B, 1, T, 96, 96\rangle$ normalised crops and a frame mask. **Operations.** 3D convolutions apply kernels that extend over three consecutive frames as well as over space. A single kernel can therefore respond to a pattern such as "lips closing", which no single frame shows; this is how the encoder captures the motion that distinguishes otherwise similar mouth shapes [1, 2]. Spatial pooling progressively removes position detail, and the BiLSTM integrates evidence across the whole utterance, using both past and future frames. **Output.** $\mathbf{H}_v \in \mathbb{R}^{B\times T\times 512}$. **Integration.** Passed to the fusion module; also used alone by the visual-only baseline. **Testing.** Shape tests (Section 5); a visual-only model must overfit a batch of 8 utterances to near-zero CTC loss.

7.2 Audio encoder

Purpose. Convert acoustic features into one vector per video frame describing the sounds being produced. **Input.** $\langle B, T_a, 80\rangle$ standardised log-Mel features and lengths. **Operations.** Two strided 1-D convolutions reduce 100 fps features to 25 fps (Section 6.3); a two-layer BiLSTM adds utterance-level context. **Output.** $\mathbf{H}_a \in \mathbb{R}^{B\times T\times 512}$. **Integration.** Passed to fusion; used alone by the audio-only baseline. **Testing.** Shape and length tests (Section 6); overfitting test as for the visual encoder.

7.3 Feature alignment and fusion

Temporal alignment. The two streams have different native rates: 25 fps for video and 100 fps for audio features. Rather than resampling inside the fusion module, the audio encoder subsamples by exactly four, so both encoders emit sequences of the same length T on the same 40 ms time grid. Frame t of $\mathbf{H}_a$ and frame t of $\mathbf{H}_v$ therefore describe the same instant. If the two lengths differ by one or two frames (rounding at clip boundaries), the longer sequence is truncated to the shorter. Cross-attention itself does not require equal lengths, but the per-frame gate below does.

Projection. Each stream is projected to a shared width $d = 256$:

$$\mathbf{A} = \mathrm{LN}(\mathbf{H}_a\mathbf{W}_a), \qquad \mathbf{V} = \mathrm{LN}(\mathbf{H}_v\mathbf{W}_v), \qquad \mathbf{A}, \mathbf{V} \in \mathbb{R}^{T\times d}, \tag{7}$$

where LN is layer normalisation (batch index omitted).

Cross-modal attention. Each stream queries the other. For audio attending to video, the *queries* come from the audio sequence and the *keys* and *values* from the video sequence:

$$\mathbf{Q} = \mathbf{A}\mathbf{W}_Q, \quad \mathbf{K} = \mathbf{V}\mathbf{W}_K, \quad \mathbf{U} = \mathbf{V}\mathbf{W}_U, \tag{8}$$

$$\boldsymbol{\alpha} = \mathrm{softmax}\Big(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} + \mathbf{M}\Big), \qquad \mathrm{Attn}(\mathbf{A}, \mathbf{V}) = \boldsymbol{\alpha}\,\mathbf{U}, \tag{9}$$

$$\tilde{\mathbf{A}} = \mathrm{LN}\left(\mathbf{A} + \mathrm{MHA}(\mathbf{A}, \mathbf{V})\right), \qquad \tilde{\mathbf{V}} = \mathrm{LN}\left(\mathbf{V} + \mathrm{MHA}(\mathbf{V}, \mathbf{A})\right). \tag{10}$$

Row t of the attention weights $\boldsymbol{\alpha} \in \mathbb{R}^{T\times T}$ says how strongly audio frame t draws on each video frame; $\mathbf{M}$ sets padded positions to $-\infty$. MHA denotes the multi-head version with four heads of width $d_k = 64$, implemented with `nn.MultiheadAttention`. The audio stream can thereby borrow visual evidence for an ambiguous sound, and the visual stream can borrow acoustic evidence for an ambiguous lip shape—the mechanism proposed for disambiguating homophenes [2].

Gated fusion. A learned gate decides, frame by frame, how much each enriched stream contributes:

$$g_t = \sigma\big(\mathbf{w}_g^\top[\tilde{\mathbf{a}}_t; \tilde{\mathbf{v}}_t] + b_g\big) \in (0, 1), \qquad \mathbf{f}_t = g_t\,\tilde{\mathbf{a}}_t + (1 - g_t)\,\tilde{\mathbf{v}}_t, \tag{11}$$

where $[\cdot\,;\cdot]$ denotes concatenation and σ the logistic function. A small g_t means that the fused representation relies mainly on the video-anchored stream. The fused sequence $\mathbf{F} = (\mathbf{f}_1, \ldots, \mathbf{f}_T)$ passes through a one-layer BiLSTM and the CTC output layer.

How noise adaptation is encouraged. The proposal states that fusion should give more weight to the lips when the audio is noisy **[Proposal]**. An attention module does not detect noise by itself; it learns whatever weighting reduces the training loss on the data it sees. Three mechanisms are possible:

(a) *explicit SNR conditioning*, feeding the true SNR to the gate;

(b) *a learned noise-estimation signal*, an auxiliary head trained to predict the SNR from the audio encoder's output;

(c) *implicit learning*, training the attention and gate on a mixture of clean and noisy examples so that they learn to down-weight unreliable audio.

This plan adopts **(c)** as the primary mechanism **[Recommended]**. Option (a) is not usable in deployment, because the true SNR of live audio is unknown. Training on all five SNR conditions (Section 4.7) exposes the gate to examples in which the audio is uninformative, and modality dropout (Section 7.4) adds examples in which it is absent. Option (b) is kept as an experiment **[To be decided]**: an auxiliary linear head on the time-averaged audio encoding regresses the

known training SNR (with "clean" mapped to 30 dB), its prediction is appended to the gate input in Eq. (11), and its loss is added to the CTC loss with a small weight such as 0.1.

How noise adaptation is evaluated. Whether the gate really behaves as intended is an empirical question. During testing the evaluation script records the mean gate value $\bar{g}$ for every utterance and reports its distribution at each SNR. If the mechanism works, $\bar{g}$ should decrease as SNR falls, and the fused model's advantage over the audio-only baseline should grow at 0 dB and -5 dB. Because $\tilde{\mathbf{a}}_t$ already contains some visual information through cross-attention, g_t is only an approximate measure of reliance on audio; the modality ablations of Section 9 (removing one stream at test time) provide the more direct evidence.

7.4 Modality dropout

Purpose. Audio is usually far more informative than video, so a fused model can learn to ignore the visual stream entirely. Modality dropout forces it to make use of both streams by occasionally removing one during training [6].

Implementation. For each training utterance a value $u \sim \mathcal{U}(0, 1)$ is drawn:

$$\text{stream used} = \begin{cases} \text{video only (audio dropped)} & u < p_a, \\ \text{audio only (video dropped)} & p_a \le u < p_a + p_v, \\ \text{both} & \text{otherwise.} \end{cases} \tag{12}$$

Both streams are never dropped together. A dropped stream's encoder output is replaced by a learned "missing modality" vector $\mathbf{e}_{\text{miss}} \in \mathbb{R}^{256}$ repeated over all frames, rather than by an attention mask. Masking every key of a sequence would make the softmax in the cross-attention undefined (all logits $-\infty$) and produce NaN values; a learned vector avoids this and lets the gate learn to recognise the missing stream.

Rate selection. Starting values are $p_a = 0.25$ and $p_v = 0.15$ **[Recommended]**; audio is dropped more often because the risk being addressed is under-use of video. The rates are tuned on validation WER averaged over all five SNR conditions, using the grid $p_a, p_v \in \{0, 0.1, 0.25, 0.4\}$ with $p_a + p_v \le 0.5$ if compute permits. The setting $p_a = p_v = 0$ is the "no modality dropout" ablation.

Inference behaviour. Modality dropout is disabled at inference. Both streams are used whenever available. If the face is not found in more than half of a window's frames, the visual encoding is replaced by $\mathbf{e}_{\text{miss}}$— the same representation the model saw during training—so the system degrades to audio-only recognition instead of being fed meaningless crops.

7.5 Sequence prediction and decoding

Why CTC. The training data provide a transcript for each utterance but not the frame at which each character is spoken. Connectionist Temporal Classification [11] removes the need for such frame-level labels by summing over every possible alignment between the T output frames and the L target characters, as used in the CTC-based lipreading and AVSR systems cited in the proposal [4, 6].

Vocabulary, blank and collapsing. The output layer predicts a distribution over the 28 symbols of Eq. (1) at every frame. CTC introduces the *blank* symbol (index 0) to mean "no new character". A frame-level path $\boldsymbol{\pi} = (\pi_1, \ldots, \pi_T)$ is mapped to text by a function $\mathcal{B}$ that first merges consecutive repeated symbols and then removes blanks, for example $\mathcal{B}(\texttt{bb_ii_nn__}) = \texttt{bin}$ and $\mathcal{B}(\texttt{gr_e_e_n}) = \texttt{green}$. A doubled letter therefore requires a blank between its two copies.

Loss. With y_k^t the predicted probability of symbol k at frame t, the probability of transcript $\mathbf{z}$ given input $\mathbf{x}$ and the CTC loss are

$$p(\mathbf{z} \mid \mathbf{x}) = \sum_{\boldsymbol{\pi} \in \mathcal{B}^{-1}(\mathbf{z})} \prod_{t=1}^{T} y_{\pi_t}^t, \qquad \mathcal{L}_{\text{CTC}} = -\log p(\mathbf{z} \mid \mathbf{x}). \tag{13}$$

The sum is computed efficiently by the forward–backward algorithm. The implementation uses PyTorch's built-in loss, `nn.CTCLoss(blank=0, zero_infinity=True)`, called with log-probabilities of shape $\langle T, B, 28 \rangle$, the concatenated target indices, the per-utterance encoder lengths $T^{(b)}$ and target lengths $L^{(b)}$. The `zero_infinity` flag prevents an occasional impossible alignment from producing an infinite loss that would corrupt training.

Conditions for CTC to work. An alignment exists only if

$$T^{(b)} \;\geq\; L^{(b)} + R^{(b)}, \tag{14}$$

where $R^{(b)}$ is the number of adjacent repeated characters in the target (each needing an extra blank). GRID transcripts contain at most about 35 characters including spaces, while a 3 s clip gives $T = 75$ encoder frames, so the condition holds with a wide margin. It would fail if the audio encoder subsampled more aggressively or if live windows were made very short; the data loader therefore asserts Eq. (14) for every item, and the window length of the live demo is chosen with it in mind. Encoder lengths must also be computed from the true (unpadded) input lengths using the convolution length formula of Section 6.3; passing padded lengths is a common silent error.

Greedy decoding. Taking the most probable symbol at each frame and applying $\mathcal{B}$ is fast and is used for validation during training.

Beam-search decoding. The proposal specifies beam search for final decoding **[Proposal]**. CTC prefix beam search keeps the W most probable *text prefixes*, tracking for each prefix separately the probability of paths ending in a blank and in a non-blank symbol, so that paths that collapse to the same text are merged. A beam width of $W = 10$ is the starting value **[Recommended]**. With only 28 symbols the algorithm is implemented directly in about 60 lines of Python, which keeps it transparent and testable.

Language-model integration. An external language model could be added by rescoring beam prefixes. On GRID this needs care: the fixed six-slot grammar means that a grammar-aware language model would correct many errors by construction, inflating accuracy in a way that would not carry over to real captioning. Headline results will therefore be reported *without* a

language model **[Recommended]**. A grammar-constrained decoder may be reported separately, clearly labelled, as an upper reference **[To be decided]**.

7.6 Forward pass and decoding algorithms

Algorithm 2 LipScribe forward pass (training and inference).

Input: mouth crops X_v $\langle B, 1, T, 96, 96\rangle$, log-Mel features X_a $\langle B, T_a, 80\rangle$, lengths $T_a^{(b)}$, flag *training*

Output: log-probabilities $\mathbf{P}$ $\langle T, B, 28\rangle$, encoder lengths $T^{(b)}$, gate values g

$\mathbf{H}_v \leftarrow \textsc{VisualEncoder}(X_v)$ ▷ $\langle B, T, 512\rangle$

$\mathbf{H}_a \leftarrow \textsc{AudioEncoder}(X_a, T_a^{(b)})$; $T^{(b)} \leftarrow \lceil T_a^{(b)}/4 \rceil$ ▷ $\langle B, T, 512\rangle$

$\mathbf{A} \leftarrow \mathrm{LN}(\mathbf{H}_a\mathbf{W}_a)$; $\mathbf{V} \leftarrow \mathrm{LN}(\mathbf{H}_v\mathbf{W}_v)$

if *training* **then** apply modality dropout: replace $\mathbf{A}$ or $\mathbf{V}$ by $\mathbf{e}_{\text{miss}}$ per sample

else if face missing in window **then** $\mathbf{V} \leftarrow \mathbf{e}_{\text{miss}}$

end if

$\tilde{\mathbf{A}} \leftarrow \mathrm{LN}(\mathbf{A} + \mathrm{MHA}(\mathbf{A}, \mathbf{V}, \text{mask}))$; $\tilde{\mathbf{V}} \leftarrow \mathrm{LN}(\mathbf{V} + \mathrm{MHA}(\mathbf{V}, \mathbf{A}, \text{mask}))$

$g \leftarrow \sigma(\mathbf{w}_g^\top[\tilde{\mathbf{A}}; \tilde{\mathbf{V}}] + b_g)$; $\mathbf{F} \leftarrow g \odot \tilde{\mathbf{A}} + (1-g) \odot \tilde{\mathbf{V}}$

$\mathbf{Z} \leftarrow \textsc{BiLSTM}(\mathbf{F}, T^{(b)})$; $\mathbf{P} \leftarrow \log_\mathrm{softmax}(\mathbf{Z}\mathbf{W}_o + \mathbf{b}_o)$, transposed to time-first

return $\mathbf{P}, T^{(b)}, g$

Algorithm 3 CTC prefix beam search for one utterance (probabilities in log space in practice).

Input: frame posteriors y_k^t for $t = 1..T$, beam width W, blank index 0

Output: most probable transcript

$\mathcal{P} \leftarrow \{\varepsilon\}$; $p_b(\varepsilon) \leftarrow 1$, $p_{nb}(\varepsilon) \leftarrow 0$

for $t = 1$ **to** T **do**

  initialise p'_b, p'_{nb} to 0 for all prefixes

  for each prefix $\ell \in \mathcal{P}$ **do**

    $p'_b(\ell)$ += $y_0^t\,(p_b(\ell) + p_{nb}(\ell))$ ▷ emit blank

    for each symbol $k \neq 0$ **do**

      $\ell^+ \leftarrow \ell \| k$

      if k = last symbol of ℓ **then**

        $p'_{nb}(\ell^+)$ += $y_k^t\, p_b(\ell)$ ▷ repeat needs a blank in between

        $p'_{nb}(\ell)$ += $y_k^t\, p_{nb}(\ell)$ ▷ repeat collapses

      else

        $p'_{nb}(\ell^+)$ += $y_k^t\,(p_b(\ell) + p_{nb}(\ell))$

      end if

    end for

  end for

  $\mathcal{P} \leftarrow$ the W prefixes with largest $p'_b + p'_{nb}$; $p_b, p_{nb} \leftarrow p'_b, p'_{nb}$

end for

return $\arg\max_{\ell \in \mathcal{P}} (p_b(\ell) + p_{nb}(\ell))$ mapped to characters

8 Training Procedure and Experiment Management

8.1 Training workflow

Training is driven by a single script, `train.py`, that reads one YAML configuration file and performs the following steps.

1. **Set up.** Load the configuration, set all random seeds, create a run directory named after the experiment and timestamp, and copy the configuration and Git commit hash into it.
2. **Build datasets.** Filter `manifest.csv` by the speakers listed for each partition in `splits.csv`. The training dataset returns, per utterance, the cached mouth crops, the clean waveform mixed with noise at a randomly drawn SNR, and the encoded transcript. Validation and test datasets read the fixed pre-mixed noisy files.
3. **Build batches.** A custom collate function pads waveforms and crop sequences, computes feature and encoder lengths, builds padding masks and concatenates targets for CTC (Section 6). Log-Mel extraction and normalisation run on the GPU on the padded batch.
4. **Initialise the model.** Convolution and linear layers use PyTorch's default (Kaiming-uniform) initialisation; LSTM forget-gate biases are set to 1 to ease gradient flow early in training; the CTC output bias for the blank symbol is left at zero.
5. **Train.** For each batch: forward pass (Algorithm 2), CTC loss (Eq. (13)) averaged over target lengths, backward pass with automatic mixed precision (AMP), gradient-norm clipping, optimiser step and scheduler step.
6. **Validate.** After each epoch, compute validation CTC loss and greedy-decoded WER and CER at every SNR, and their mean.
7. **Checkpoint.** Save the latest state and, separately, the best state by mean validation WER. A checkpoint contains model, optimiser, scheduler and AMP scaler states, the epoch, the configuration and the vocabulary.
8. **Stop.** End training when mean validation WER has not improved for the patience period, or at the maximum epoch count.
9. **Restore and export.** Reload the best checkpoint, run the final evaluation (Section 9) and export the model weights for the demonstration.

8.2 Initial hyperparameters

Table 10 lists starting values **[Recommended]**. They are common defaults for models of this size and are *not* established optimal settings for LipScribe; the most sensitive values (learning rate, dropout and modality-dropout rates) will be tuned on the validation set.

Table 10: Initial training hyperparameters (starting values to be tuned).

Hyperparameter	Starting value	Rationale
Optimiser	AdamW	Robust default for recurrent and attention models
Peak learning rate	3×10^{-4}	Tuned over $\{10^{-4}, 3 \times 10^{-4}, 10^{-3}\}$
Weight decay	10^{-2}	Mild regularisation; not applied to biases or norms
Schedule	1 epoch linear warm-up, then cosine decay	Warm-up stabilises early CTC training
Batch size	16 utterances	Expected to fit an 8 GB GPU with AMP; use gradient accumulation otherwise
Gradient clipping	global norm 5.0	Prevents LSTM gradient explosions
Dropout	0.3 (encoders), 0.1 (attention)	Limits overfitting to 27 training speakers
Modality dropout	$p_a = 0.25$, $p_v = 0.15$	Section 7.4
Maximum epochs	60	Upper bound; early stopping usually ends sooner
Early-stopping patience	8 epochs	Measured on mean validation WER
Beam width (evaluation)	10	Section 7.5
Random seeds	1, 2, 3	Three seeds for the final models, if compute allows

Gradient management. LSTMs and CTC can produce occasional very large gradients, particularly early in training. Global-norm clipping bounds the update size. With AMP, gradients are unscaled before clipping. If a batch produces a non-finite loss, the step is skipped and logged; more than a handful of such events indicates a data or length bug rather than bad luck.

Curriculum for visual models. Visual-only CTC training is known to be slow to start. If the visual-only baseline has not begun to reduce validation CER after several epochs, the team will first train the visual encoder alone and then use its weights to initialise the visual stream of the fused model **[To be decided]**.

8.3 Experiment logging and management

Each run logs training loss, validation loss, WER and CER per SNR, learning rate, gradient norm and mean gate value to TensorBoard, and appends one summary row to `experiments/results.csv` on completion. Run directories are never overwritten. An experiment is reported only if its configuration file is committed, so that every number in the final report can be traced to a configuration, a commit and a checkpoint.

Listing 2: Abbreviated experiment configuration for the proposed model **[Recommended]**.

```
experiment: proposed_attention_fusion
seed: 1
data:
  manifest: data/manifest.csv
  splits: data/splits.csv
  train_snrs: [clean, 10, 5, 0, -5]
  eval_snrs: [clean, 10, 5, 0, -5]
model:
  streams: [audio, visual]
  visual_encoder: conv3d_bilstm # V1; alternative: conv3d_effnetb0
  audio_features: logmel80 # A1; alternatives: mfcc39, logmel_mfcc
  fusion: cross_attention_gated # alternatives: concat, none
  modality_dropout: {audio: 0.25, visual: 0.15}
```

```
train:
  optimiser: adamw
  lr: 3.0e-4
  batch_size: 16
  max_epochs: 60
  patience: 8
  grad_clip: 5.0
decode: {method: beam, beam_width: 10}
```

8.4 Baselines and experiment plan

The proposal requires three simple baselines—audio-only, visual-only and simple concatenation—followed by the full system with a 3D convolutional visual encoder and adaptive cross-modal attention with modality dropout **[Proposal]**. All models share the same code, encoders, data partitions, noise files, feature normalisation, optimiser, schedule, maximum epoch count and early-stopping rule. Only the components under test differ. The audio-only baseline is trained with the same multi-condition noise as the fused models; a baseline trained only on clean audio would be unrealistically weak in noise and make the comparison unfair.

- **B1 Audio-only:** audio encoder → sequence BiLSTM → CTC.
- **B2 Visual-only:** visual encoder (V1) → sequence BiLSTM → CTC.
- **B3 Simple fusion:** concatenate **A** and **V** per frame, linear projection to 256, then the same sequence BiLSTM and CTC; no attention, no modality dropout.
- **P Proposed:** the complete model of Section 7.

Table 11: Planned experiments. No results are reported; the last column states what each comparison is designed to reveal.

ID	Model	Key comparison	Diagnostic value
B1	Audio-only	Reference for all fused models	How fast ASR alone degrades as SNR falls
B2	Visual-only	Constant across SNR by design	Upper bound on what the lips alone can provide on GRID; checks the visual pipeline works
B3	Concatenation fusion	B3 vs. B1	Whether adding video helps at all without adaptive weighting
P	Attention fusion + modality dropout	P vs. B3, P vs. B1	Benefit of adaptive fusion, especially at 0 and -5 dB
X1	P without modality dropout	X1 vs. P	Contribution of modality dropout; whether video is otherwise ignored
X2	P with concatenation in place of attention, keeping dropout	X2 vs. P	Contribution of attention itself, separated from dropout
X3	P with a 2D per-frame CNN instead of the 3D front-end	X3 vs. P	Contribution of 3D (spatiotemporal) convolution
X4	P with EfficientNet-B0 trunk (V2)	X4 vs. P	Value of a pretrained 2D backbone (optional)
X5	B1 with A2 / A3 features	vs. B1	Whether MFCC features add anything to log-Mel
X6	P with SNR auxiliary head	X6 vs. P	Whether an explicit noise estimate improves gating (optional)

B1, B2, B3 and P form the core and are completed first. X1–X3 are the ablations the proposal asks for and are run next. X4–X6 are optional and depend on remaining time and compute.

9 Evaluation and Testing

9.1 Metrics

The proposal specifies Word Error Rate and Character Error Rate as the evaluation metrics **[Proposal]**. Both are based on the minimum edit distance between the recognised text (hypothesis) and the reference transcript. For a reference of N words, let S, D and I be the numbers of word substitutions, deletions and insertions in the minimum-cost alignment. Then

$$\mathrm{WER} = \frac{S + D + I}{N} \times 100\%, \qquad \mathrm{CER} = \frac{S_c + D_c + I_c}{N_c} \times 100\%, \tag{15}$$

where the subscript c denotes the same quantities counted over characters (including spaces). Corpus-level values are computed by summing errors and reference lengths over the whole test set before dividing, not by averaging per-utterance rates; this is the convention of the `jiwer` library used for the computation. WER can exceed 100% when there are many insertions. CER is the more sensitive metric for GRID's single-letter slot, where a single wrong character makes a whole word wrong.

9.2 Evaluation protocol

1. Load the best checkpoint of each model (selected on validation data only).
2. For each condition $\rho \in \{\text{clean}, 10, 5, 0, -5\}$ dB, decode every test utterance of the four held-out speakers with beam search, using the fixed pre-mixed noisy audio.
3. Compute corpus-level WER and CER per condition, per speaker and in total, and record the mean fusion gate per utterance.
4. Repeat for every model in Table 11 on identical inputs.
5. Where three seeds were trained, report mean and standard deviation.

Performance is reported separately for each SNR, because averaging across noise levels would hide the effect the project is designed to measure: how quickly accuracy degrades as noise increases and where the visual stream begins to help. The main result will be a WER-versus-SNR plot with one line per model and a matching table, as sketched in Table 12.

Table 12: Template for reporting test WER (%). Cells are intentionally empty: no results have been obtained yet.

Model	Clean	10 dB	5 dB	0 dB	−5 dB
B1 Audio-only	–	–	–	–	–
B2 Visual-only	–	–	–	–	–
B3 Concatenation	–	–	–	–	–
P Proposed	–	–	–	–	–

9.3 Comparisons and their interpretation

- **Overall quality.** The proposal sets an expected target of WER below 10% for the fused model under clean conditions **[Proposal]**. This is a project target, not a guaranteed outcome; the evaluation will report whether it is met.
- **Degradation with noise.** The slope of WER against SNR for each model shows how robust it is. The proposal expects a clear advantage over the audio-only baseline at 0 and −5 dB **[Proposal]**.
- **Versus audio-only (P vs. B1).** The primary test of the project's hypothesis.
- **Versus visual-only (P vs. B2).** At −5 dB the fused model should be no worse than the visual-only model, since in the limit it can ignore the audio; if it is worse, fusion is being misled by noisy audio.
- **Versus simple fusion (P vs. B3).** Isolates the benefit of attention and modality dropout over naive combination.
- **Speaker independence.** All numbers are on unseen speakers. Per-speaker WER is also reported, because a mean can hide one speaker on whom the system fails.
- **Statistical caution.** With four test speakers, differences of a fraction of a percentage point may not be meaningful. Where seeds allow, differences are reported with their seed-

to-seed spread, and a paired bootstrap over test utterances is used to give a confidence interval for the most important comparison (P vs. B1 at each SNR) **[Recommended]**.

If the fused model is worse than audio-only. This can happen, particularly on clean audio, where video adds little information but adds parameters and noise. The team will check, in order: (1) whether the visual-only model learned anything (if B2 is near chance, the visual pipeline is broken); (2) whether the gate values vary with SNR or are constant; (3) whether training loss is lower but validation loss higher than B1, indicating overfitting to training speakers' faces; (4) whether modality dropout rates are too high, starving the audio stream; and (5) whether the visual stream is misaligned in time (Section 9.6). A fused model that is slightly worse when clean but better at low SNR is still consistent with the project's goal and will be reported as such.

9.4 Ablation studies

Ablations X1–X3 (Table 11) each remove one advanced component while keeping everything else fixed: modality dropout, attention fusion and the 3D convolutional front-end. They are evaluated with the same per-SNR protocol. Two additional test-time ablations use the proposed model without retraining: replacing the visual encoding by $\mathbf{e}_{\text{miss}}$ (audio-only operation) and replacing the audio encoding by $\mathbf{e}_{\text{miss}}$ (visual-only operation). They show how much each stream contributes inside the fused model.

9.5 Latency and real-time feasibility

For the demonstration, the team will measure on the demonstration machine:

- per-frame Face Mesh and crop time;
- model forward time and beam-search time for one window;
- end-to-end caption latency, from the end of a spoken phrase to the appearance of its caption;
- the real-time factor RTF $= t_{\text{processing}}/t_{\text{audio}}$.

An RTF below 1 is necessary for live operation; the target for the demonstration is an RTF of at most 0.5 and caption latency of about 3–4 s **[Recommended]** (Section 10). Timings are reported as the median and 95th percentile over at least 100 windows, after a warm-up period.

9.6 Software testing

Tests are written with `pytest` and run before every merge to the main branch. Small fixture files (two GRID utterances and one MUSAN noise file) are kept in the repository so that tests run without the full dataset.

Table 13: Representative automated tests and acceptance criteria.

Level	Test	Assertion / acceptance criterion
Unit	Tokeniser round trip	`decode(encode(s)) == s` for all GRID word strings; blank never produced by `encode`
Unit	CTC collapse	`collapse([b,b,0,i,0,n,n]) == "bin"`; `collapse([e,0,e]) == "ee"`
Unit	Noise mixing	measured SNR within 0.01 dB of target for all five conditions; output length equals input length; peak ≤ 1
Unit	Mouth crop	output shape $(96, 96)$, dtype `uint8`; failed detection yields interpolated box and invalid flag
Unit	Feature extraction	3 s waveform gives $\langle 300, 80 \rangle$; matches librosa within tolerance
Unit	Encoder shapes	visual and audio encoders map example inputs to $\langle B, 75, 512 \rangle$
Unit	Metrics	WER(`"bin blue at f two now"`, `"bin blue at s two now"`) $= 1/6$
Unit	Beam search	with $W = 1$, beam search equals greedy decoding on random posteriors
Unit	Splits	train, validation and test speaker sets are pairwise disjoint
Integration	Data loader	one batch has consistent lengths; every item satisfies Eq. (14)
Integration	Overfit check	each model reaches CTC loss < 0.05 and CER $= 0$ on 8 utterances within 300 steps
Integration	Train/infer parity	live preprocessing of a GRID file equals cached features (max error $< 10^{-5}$)
Integration	Synchronisation	shifting video by 5 frames relative to audio increases validation loss of the trained model (sanity check of alignment)
End-to-end	Inference script	`transcribe.py` on a test clip returns a non-empty lower-case string within the latency budget
End-to-end	Demo smoke test	demo starts, processes a recorded clip and displays a caption without error

10 Application Integration and Live Captioning

10.1 Batch prototype versus streaming application

Two kinds of application are possible, and the distinction determines how much engineering the demonstration needs.

- A **batch prototype** processes a complete recorded clip—uploaded or recorded in the browser—and returns its caption. It reuses the evaluation code almost unchanged and its correctness is easy to verify.
- A **streaming application** captions speech while it is being spoken. It needs buffering, synchronisation of two live devices, and repeated inference on overlapping windows. Because the encoders are bidirectional, the model needs future context and cannot emit a character the moment a frame arrives; streaming is therefore approximated by re-decoding recent windows, which introduces latency of roughly one window.

The recommended plan **[Recommended]** is to build the batch prototype first (Mode A) and then a windowed pseudo-streaming mode (Mode B) that reuses its preprocessing and inference functions. Mode A is the guaranteed deliverable; Mode B is the live captioning demonstration.

A fully streaming model with unidirectional encoders is outside the six-week scope.

10.2 Mode A: batch captioning in Gradio

A Gradio page offers a webcam-and-microphone recorder and a file-upload box. On submission, the server extracts frames with OpenCV and audio with `ffmpeg`, resamples video to 25 fps and audio to 16 kHz, runs the visual and audio pipelines, the model and beam search, and displays the caption beside the video. The page also shows the mouth-crop sequence and a plot of the per-frame fusion gate, which makes the system's behaviour visible during the presentation. An optional control adds MUSAN noise at a chosen SNR to the clip, so that the audience can see captions with and without noise on the same recording.

10.3 Mode B: windowed live captioning

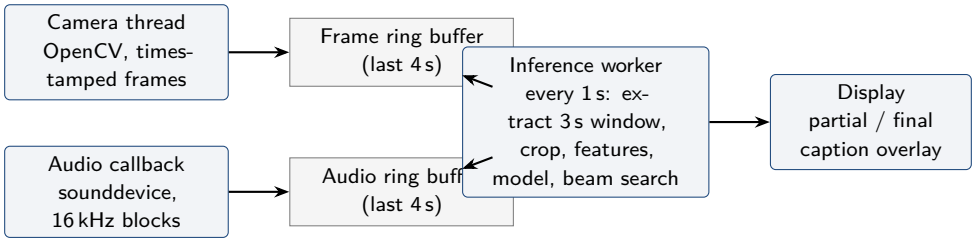


Figure 3: Threads and buffers of the windowed live-captioning mode.

Input and buffering. A camera thread reads frames with OpenCV and stores each with a timestamp from a single monotonic clock. A `sounddevice` callback appends 16 kHz audio blocks, also timestamped, to a ring buffer. Both buffers hold the most recent four seconds, which bounds memory use (four seconds of 640×480 colour frames at 30 fps is about 110 MB; the buffer can store downscaled frames or pre-computed crops to reduce this).

Temporal synchronisation. Webcams rarely deliver exactly 25 fps and may drop frames. For a window covering times $[t_0, t_0 + 3\,\text{s})$ the worker selects, for each of the 75 target instants $t_0 + 40k$ ms, the buffered frame with the nearest timestamp, and cuts the matching 48,000 audio samples. Any constant offset between the camera and microphone pipelines is measured once with a hand-clap test (comparing the frame where the hands meet with the audio onset) and subtracted **[Recommended]**.

Windowing and preprocessing. Every second the worker processes the latest 3 s window, matching the duration of GRID training utterances and giving $T = 75$, which satisfies the CTC length condition (Eq. (14)). Face Mesh runs on each new frame as it arrives rather than once per window, so each frame is processed only once even though windows overlap; the crop smoothing uses an exponential moving average over past frames. Audio features are computed with the stored training statistics.

Inference and decoding. The model runs in evaluation mode, with gradient tracking disabled by `torch.inference_mode()`, and its output is decoded by beam search. If more than half of the window's frames lack a face, the visual stream is replaced by $\mathbf{e}_{\text{miss}}$ and the caption is marked "audio only".

Partial and final captions. A simple energy-based voice-activity detector tracks speech on the audio buffer. While speech continues, the caption for the latest window is shown as a grey *partial* caption and replaced each second. When the detector reports at least 500 ms of silence, the worker decodes one window aligned to the detected utterance and displays it as a *final* caption in black, appended to a short caption history. Restricting final captions to detected utterances avoids the difficult problem of merging text from overlapping windows.

Errors and missing input. If the camera or microphone cannot be opened, the application reports it and continues in the remaining modality where possible. Exceptions in the inference worker are logged and the window is skipped, so a single failure does not stop the display loop.

10.4 Latency, cost and memory

Caption latency in Mode B is the sum of waiting for the window to fill (up to 1 s for partial captions, the end-of-speech delay for final ones), Face Mesh on the latest frames, and model and beam-search time. The 3D convolutions over 75 frames dominate model cost; the per-window cost is the same as one test utterance and will be measured as described in Section 9. Real-time audio-visual recognition on modest hardware is still an active research problem [8, 3], so these measurements, rather than assumptions, will decide the final window settings. If latency is too high, the team will, in order: run inference every 1.5 s instead of 1 s, reduce beam width, run the model on a GPU, or quantise the model weights.

10.5 Testing the application

- **Pre-recorded clips.** Held-out GRID test clips are played through Mode A and through Mode B (by feeding the file to the buffers in real time). Their captions must match those produced by the evaluation script for the same clips, which confirms that the application path is equivalent to the evaluated path.
- **Live input.** Team members read GRID-grammar sentences to the webcam in a quiet room and with played-back background noise. Because the model is trained on GRID, it is expected to work only on GRID-like sentences spoken in similar framing; free conversation is outside its training distribution, and the demonstration will say so.
- **Failure cases.** Covering the mouth, turning the head and leaving the frame must produce "audio only" captions rather than crashes.

11 Additional Objectives: Emotion Detection and Language Identification

The proposal includes emotion detection (objective 3) and spoken-language identification (objective 4) **[Proposal]**. Neither can be trained on GRID alone: GRID has no emotion labels, and all of its sentences are in English with one fixed grammar [9]. Both are therefore planned as *separate, optional modules* with their own data, models and evaluation. They share preprocessing code with the core system but do not change the speech-recognition architecture or its results, and work on them begins only after the core milestones in Section 12 are met.

11.1 Emotion detection from facial movement

Approach. The literature reviewed in the proposal selects Face Mesh landmarks that correspond to Facial Action Coding System regions and converts them into compact geometric feature vectors, with reported classification accuracies of up to 97% on their data [7]. LipScribe would follow the same route **[To be decided]**:

1. reuse the Face Mesh landmarks already computed by the visual pipeline;
2. compute geometric features per frame—normalised distances (mouth width and opening, eyebrow-to-eye distance, eye aperture) and angles between landmark triples—after aligning faces to remove head pose and scale;
3. summarise their temporal changes over an utterance with statistics (mean, range, velocity) or a small LSTM;
4. classify with a lightweight model (for example a random forest or multilayer perceptron from scikit-learn) into a small set of emotion categories.

Data required. An emotion-labelled audio-visual dataset is required. Candidates include CREMA-D [16] and RAVDESS [17], both of which provide acted emotional speech with frontal video; the team would confirm licence terms before use. Evaluation would again use speaker-disjoint splits and report per-class accuracy and a confusion matrix.

Limitations. Facial movement while speaking is dominated by articulation, so mouth landmarks partly reflect *what* is said rather than how the speaker feels. Acted emotions in such datasets are more exaggerated than natural ones, facial expression is not a reliable indicator of internal emotional state, and expressions vary across individuals and cultures. Any emotion output would be presented as an experimental label, not as a statement about the user.

Integration, if completed. A separate classification head operating on landmark features, displayed as a tag next to each final caption. It would not share loss or weights with the speech recogniser.

11.2 Spoken-language identification

Approach. Language identification predicts which language an utterance is spoken in. A natural design would add a separate utterance-level classification head to the existing encoders: time-pool the fused representation $\mathbf{F}$ and apply a linear classifier over the candidate languages, trained with cross-entropy alongside or after CTC training **[To be decided]**. Audio carries most of the evidence; the proposal's interest is whether lip movements add to it, since visual speech patterns differ between languages [3, 4].

Data required. Multilingual audio-visual data with language labels is essential, and recognising speech (not just identifying the language) in each additional language would need transcribed data and an extended character vocabulary. Candidate sources are the multilingual MuAViC corpus [18] or language-specific corpora such as those used for Indonesian and Kannada lipreading [3, 4], subject to availability and licensing. A locally recorded set of short phrases in English

and a Ugandan language could serve as a small qualitative test, with participants' informed consent.

Limitations. A model trained on GRID has seen one language, a 51-word vocabulary and studio conditions. It cannot be expected to transfer to other languages, accents or scripts without new training data. Language identification on short utterances is also confounded by speaker identity if each language is spoken by different people, so speaker-disjoint evaluation is again required.

11.3 Summary of scope

Table 14: Core deliverable versus optional extensions.

Capability	Data	Model change	Status
Audio-visual speech recognition	GRID + MUSAN	Full model (Section 7)	Core
Emotion detection	Emotion-labelled AV corpus (e.g. CREMA-D, RAVDESS)	Separate landmark-feature classifier	Optional
Language identification	Multilingual AV corpus with language labels	New utterance-level head; new vocabulary for multilingual ASR	Optional

12 Implementation Schedule and Milestones

The schedule covers the six-week capstone period and is ordered by dependency: no model can be trained before the data are verified, and no fusion experiment is meaningful before both single-modality baselines work. Training runs are long, so they are started as early as possible and overlap with development of the next component. The optional extensions are scheduled only in slack time after the core system is complete. Figure 4 shows the timeline and Table 15 the deliverables and completion criteria of each milestone.

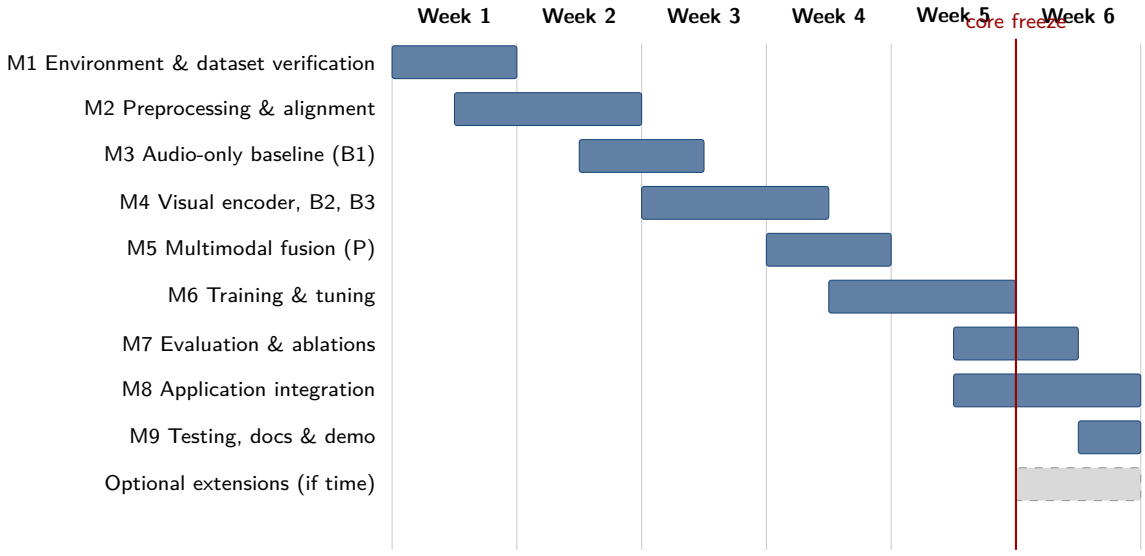


Figure 4: Gantt-style implementation timeline. Dashed bar: optional work, started only after the core system is frozen at the end of week 5.

Table 15: Milestones, dependencies, deliverables and completion criteria.

ID	Milestone	Needs	Deliverables	Completion criteria
M1	Environment setup and dataset verification	–	Repository skeleton, pinned environment, raw GRID and MUSAN, integrity report, `splits.csv`	Every member runs the test suite; integrity report lists usable utterances per speaker; speaker sets disjoint; GPU access confirmed
M2	Preprocessing and data alignment	M1	Cached mouth crops, 16 kHz audio, manifest, noise mixer, fixed noisy test sets, normalisation statistics, data loader	Crop contact sheets reviewed; noise-mixing tests pass; one batch loads with consistent lengths; train/infer parity test passes
M3	Baseline implementation (audio-only)	M2	Training and evaluation scripts, B1 checkpoint	B1 overfits 8 utterances; full B1 run completes and is evaluated at all five SNRs
M4	Visual and audio encoder development	M2	Visual encoder V1, B2 and B3 checkpoints	Encoder shape tests pass; B2 validation CER falls clearly below that of an untrained model; B3 evaluated at all SNRs
M5	Multimodal fusion	M3, M4	Attention fusion, gate, modality dropout, first P checkpoint	Fusion unit tests pass; no NaN under full-modality dropout; P trains stably
M6	Training and hyperparameter tuning	M5	Tuned P configuration; final checkpoints of B1–B3 and P	Learning-rate and modality-dropout sweeps logged; best configuration chosen on validation data only
M7	Evaluation and ablation studies	M6	Per-SNR WER/CER tables and plots, ablations X1–X3, gate analysis, latency measurements	All core models evaluated on identical test audio; every number traceable to a config and checkpoint
M8	Application integration	M3 (Mode A), M6 (final)	Gradio Mode A, live Mode B, `transcribe.py`	Mode A captions match the evaluation script on test clips; Mode B runs for 5 minutes without error
M9	Testing, documentation and demonstration	M7, M8	Final report, README, demonstration rehearsal, frozen release tag	Fresh clone installs and reproduces one evaluation; demonstration rehearsed with a recorded fallback clip

Prioritisation. If the schedule slips, work is dropped in this order: optional extensions; experiments X4–X6; Mode B live captioning (Mode A remains); multiple seeds. Baselines B1–B3, the proposed model P, the per-SNR evaluation and ablations X1–X3 are never dropped, because they constitute the core contribution described in the proposal.

Team organisation. Work is divided into three pairs **[Recommended]**: a data pair (M1–M2, noise and integrity), a model pair (encoders, fusion, training) and an application pair (inference script, Gradio interface, live mode, tests), with all members contributing to evaluation and the report. A short weekly review checks completion criteria and re-plans.

13 Risks, Limitations and Mitigation

Table 16 lists the main implementation risks, an assessment of their likelihood and impact, and the planned mitigation. Assessments are the team's judgement at the planning stage and will be reviewed weekly.

Table 16: Implementation risks and mitigation strategies (L = likelihood, I = impact).

Risk	L / I	Mitigation
Limited compute resources	High / High	Confirm GPU access in week 1; use mixed precision and cached crops; develop on a subset of speakers; keep the model moderate in size; drop optional experiments first (Section 12).
Long training times	High / Medium	Start B1 in week 2 and overlap training with development; use early stopping; schedule long runs overnight; checkpoint every epoch so runs can be resumed.
Misaligned audio and video	Medium / High	Never drop frames during cropping; integrity check of durations; automated train/infer parity test; time-shift sanity test (Table 13); clap-test calibration for live input.
CTC training instability	Medium / High	Learning-rate warm-up, gradient clipping and `zero_infinity`; assert the length condition (Eq. (14)); overfit a tiny batch before full runs; log non-finite losses.
Token distribution issues	Medium / Low	The blank symbol dominates CTC outputs and GRID's grammar makes some characters much more frequent; monitor CER per word slot and the share of blank predictions; inspect the letter slot separately, as it is the hardest.
Speaker leakage	Low / High	Speaker-level split stored in one file; loader refuses overlapping speakers; disjointness unit test; statistics from training speakers only.
Poor generalisation to unseen speakers	Medium / High	Crop jitter, horizontal flip, dropout and weight decay; model selection on held-out validation speakers; report per-speaker results honestly.
Mouth detection failure (lighting, head pose, occlusion)	Medium / Medium	GRID is frontal and well lit, so this mainly affects the live demo; interpolate short gaps; fall back to audio-only captions using $\mathbf{e}_{\text{miss}}$; good lighting and frontal camera position for the demonstration.
Attention fusion fails to improve performance	Medium / High	Report the outcome truthfully; diagnose with gate analysis, test-time modality ablations and the checklist in Section 9; tune modality dropout; B3 remains a working fused system.
Excessive inference latency	Medium / Medium	Measure early on the demo machine; increase window hop, reduce beam width, use a GPU or quantise; Mode A batch captioning as the guaranteed fallback.
Limited multilingual generalisation	High / Low (core)	Treat language identification as an optional extension with its own data; state clearly that the core model covers GRID English only.
Domain gap between GRID and live speech	High / Medium	Restrict the demonstration to GRID-grammar sentences; explain the closed vocabulary; identify larger datasets (e.g. those in [6]) as future work.

Privacy and input	of camera microphone	Medium / High	Process all input locally; do not store live video or audio by default; show a clear on-screen indicator while capturing; record any demonstration data only with written consent and delete it after the project; never upload live input to third-party services.

13.1 Limitations of the planned system

Even if every risk is managed, the system will have known limitations. GRID's fixed grammar and 51-word vocabulary [9] make the recognition task far easier than open captioning, so a low WER on GRID does not imply comparable accuracy on lectures or meetings [6]. Synthetic additive noise does not reproduce reverberation, competing speakers or microphone distortion. The visual front-end assumes a roughly frontal, unoccluded face, which the literature identifies as an open problem [5, 4]. The final report will state these limitations alongside the results.

14 Expected Deliverables

The proposal commits to a clean inference script and a simple web demonstration **[Proposal]**. Table 17 lists the full set of software artefacts the implementation will produce and the evidence that will show each one works.

Table 17: Software deliverables and evidence of completion.

Deliverable	Content	Evidence that it works
Modular source code	Python package `lipscribe/` with `data`, `preprocessing`, `models`, `decoding`, `evaluation` and `app` sub-packages	Package installs with `pip install -e .`; modules imported by scripts and tests
Data-preparation scripts	`verify_dataset.py`, `extract_mouths.py`, `prepare_audio.py`, `make_noisy_test.py`, `compute_stats.py`	Integrity report, manifest and contact sheets generated from raw data by one documented command sequence
Configuration files	One YAML file per experiment in `configs/`	Every reported run has a committed configuration
Training and evaluation scripts	`train.py`, `evaluate.py`, `transcribe.py`	Re-running `evaluate.py` on a released checkpoint reproduces its reported WER
Model checkpoints	Best checkpoints of B1, B2, B3 and P (and ablations)	Load without error and match logged validation scores
Baseline implementations	Audio-only, visual-only and concatenation models	Evaluated at all five SNRs with the same protocol as P
Experiment logs and outputs	TensorBoard logs, `results.csv`, per-SNR tables, WER-vs-SNR and gate plots	Each table entry traceable to a run directory
Automated tests	`pytest` suite (Table 13)	All tests pass on a fresh environment; test report included
Captioning demonstration	Gradio Mode A and live Mode B	Live demonstration plus a recorded fallback video; Mode A matches `evaluate.py` on test clips

Table 17 continued

Deliverable	Content	Evidence that it works
Setup and execution instructions	`README.md`, `requirements.txt`, data download guide	A team member not involved in setup reproduces one evaluation from a fresh clone
Final technical report	Methods, results, ablations, limitations	Submitted; all results come from logged runs

15 Conclusion

This document has translated the LipScribe proposal into an implementation plan. The system is built from separable modules: a visual stream that uses MediaPipe Face Mesh to extract stabilised 96×96 mouth crops and encodes them with a 3D convolutional front-end and a BiLSTM; an audio stream that converts 16 kHz audio into log-Mel features and encodes them with strided convolutions and a BiLSTM at the same 25 fps rate; a cross-modal attention and gating module that combines the two; and a CTC output layer decoded by prefix beam search. Modality dropout and multi-condition training with MUSAN noise are the mechanisms through which the fusion is expected to learn to rely on the lips when the audio degrades.

The architecture, the development stages and the evaluation plan are tied together by design. Each milestone produces a component with its own tests; the baselines reuse the same components, so that every comparison isolates one change; and the per-SNR evaluation on held-out speakers measures exactly the property the project claims to improve. The same preprocessing code serves training and the captioning demonstration, so the demonstrated behaviour reflects the evaluated model.

The plan also states what remains uncertain. The clean-condition target of WER below 10% and the expected advantage at 0 and -5 dB are hypotheses to be tested, not results. Emotion detection and language identification require data that GRID does not provide and are kept as optional extensions. By completing the core recogniser first, reporting outcomes faithfully—including any in which fusion does not help—and documenting every experiment, the team aims to deliver a working, verifiable audio-visual captioning system within the six-week course.

References

References from the project proposal

[1] S. A. A. Jeevakumari and K. Dey, "LipSyncNet: A novel deep learning approach for visual speech recognition in audio-challenged situations," *IEEE Access*, vol. 12, pp. 110891–110904, 2024. DOI: 10.1109/ACCESS.2024.3436931

[2] D. Li, Y. Gao, C. Zhu, Q. Wang, and R. Wang, "Improving speech recognition performance in noisy environments by enhancing lip reading accuracy," *Sensors*, vol. 23, no. 4, p. 2053, 2023. DOI: 10.3390/s23042053

[3] Aripin and A. Setiawan, "Indonesian lip-reading detection and recognition based on lip shape using face mesh and long-term recurrent convolutional network," *Applied Computational Intelligence and Soft Computing*, vol. 2024, p. 6479124, 2024. DOI: 10.1155/2024/6479124

[4] Divya and D. Suresha, "KanAVNet: A CNN-BiLSTM-CTC-based audio-visual speech recognition system for Kannada to assist the hearing impaired," *I.J. Image, Graphics and Signal Processing*, vol. 2, pp. 87–106, 2026. DOI: 10.5815/ijigsp.2026.02.06

[5] A. W. A. Ameer, P. Salehpour, and M. Asadpour, "Deep transfer learning for lip reading based on NASNetMobile pretrained model in wild dataset," *IEEE Access*, vol. 13, pp. 11625–11640, 2025. DOI: 10.1109/ACCESS.2024.3470521

[6] T. Afouras, J. S. Chung, A. Senior, O. Vinyals, and A. Zisserman, "Deep audio-visual speech recognition," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 12, pp. 8717–8727, 2022. DOI: 10.1109/TPAMI.2018.2889052

[7] A. I. Siam, N. F. Soliman, A. D. Algarni, F. E. Abd El-Samie, and A. Sedik, "Deploying machine learning techniques for human emotion detection," *Computational Intelligence and Neuroscience*, vol. 2022, p. 8032673, 2022. DOI: 10.1155/2022/8032673

[8] D. Ryumin, D. Ivanko, and E. Ryumina, "Audio-visual speech and gesture recognition by sensors of mobile devices," *Sensors*, vol. 23, no. 4, p. 2284, 2023. DOI: 10.3390/s23042284

Additional references introduced in this document

[9] M. Cooke, J. Barker, S. Cunningham, and X. Shao, "An audio-visual corpus for speech perception and automatic speech recognition," *The Journal of the Acoustical Society of America*, vol. 120, no. 5, pp. 2421–2424, 2006. DOI: 10.1121/1.2229005

[10] D. Snyder, G. Chen, and D. Povey, "MUSAN: A music, speech, and noise corpus," arXiv preprint arXiv:1510.08484, 2015.

[11] A. Graves, S. Fernández, F. Gomez, and J. Schmidhuber, "Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks," in *Proc. 23rd International Conference on Machine Learning (ICML)*, 2006, pp. 369–376. DOI: 10.1145/1143844.1143891

[12] Y. M. Assael, B. Shillingford, S. Whiteson, and N. de Freitas, "LipNet: End-to-end sentence-level lipreading," arXiv preprint arXiv:1611.01599, 2016.

[13] M. Tan and Q. V. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in *Proc. 36th International Conference on Machine Learning (ICML)*, PMLR vol. 97, 2019, pp. 6105–6114.

[14] C. Lugaresi *et al.*, "MediaPipe: A framework for building perception pipelines," arXiv preprint arXiv:1906.08172, 2019.

[15] Y. Kartynnik, A. Ablavatski, I. Grishchenko, and M. Grundmann, "Real-time facial surface geometry from monocular video on mobile GPUs," arXiv preprint arXiv:1907.06724, 2019.

[16] H. Cao, D. G. Cooper, M. K. Keutmann, R. C. Gur, A. Nenkova, and R. Verma, "CREMA-D: Crowd-sourced emotional multimodal actors dataset," *IEEE Transactions on Affective Computing*, vol. 5, no. 4, pp. 377–390, 2014.

[17] S. R. Livingstone and F. A. Russo, "The Ryerson Audio-Visual Database of Emotional Speech and Song (RAVDESS): A dynamic, multimodal set of facial and vocal expressions in North American English," *PLoS ONE*, vol. 13, no. 5, e0196391, 2018. DOI: 10.1371/journal.pone.0196391

[18] M. Anwar, B. Shi, V. Goswami, W.-N. Hsu, J. Pino, and C. Wang, "MuAViC: A multilingual audio-visual corpus for robust speech recognition and robust speech-to-text translation," in *Proc. Interspeech*, 2023. arXiv:2303.00628